Fondements informatiques I

Cours 9: Algorithmes : manipuler des tableaux

Sorina Ionica sorina.ionica@uvsq.fr

Bilal FAYE bilal.faye@uvsq.fr

Qu'est-ce qu'un algorithme ?

Un **algorithme** est une suite finie d'étapes permettant de résoudre un problème ou d'effectuer une tâche.

Exemple : analyser les notes d'un examen

On dispose d'un tableau de notes :

```
notes = [12, 8, 15, 10, 6]
```

On souhaite :

- trouver la note la plus petite ;
- trouver la note la plus grande ;
- afficher les notes dans l'ordre croissant.

Qu'est-ce qu'un algorithme ?

Un **algorithme** est une suite finie d'étapes permettant de résoudre un problème ou d'effectuer une tâche.

Exemple : analyser les notes d'un examen

On dispose d'un tableau de notes :

```
notes = [12, 8, 15, 10, 6]
```

On souhaite :

- trouver la note la plus petite ;
- trouver la note la plus grande ;
- afficher les notes dans l'ordre croissant.

Les trois éléments d'un algorithme

- **Entrée :** les données dont on dispose (ici, les notes).
- **Traitement :** les étapes à effectuer.
- **Sortie :** le résultat attendu.

Avant de coder : réfléchir à la méthode

Pour résoudre un problème, il faut d'abord réfléchir aux étapes nécessaires, avant de les traduire en Python.

Exemple : trouver la plus petite note

Comment déterminer la plus petite valeur d'un tableau de notes ?

1. Prendre la première note comme minimum provisoire.
2. Parcourir les autres notes une par une.
3. Si une note est plus petite, mettre à jour le minimum.
4. À la fin, le minimum provisoire est la plus petite note.

Avant de coder : réfléchir à la méthode

Pour résoudre un problème, il faut d'abord réfléchir aux étapes nécessaires, avant de les traduire en Python.

Exemple : trouver la plus petite note

Comment déterminer la plus petite valeur d'un tableau de notes ?

1. Prendre la première note comme minimum provisoire.
2. Parcourir les autres notes une par une.
3. Si une note est plus petite, mettre à jour le minimum.
4. À la fin, le minimum provisoire est la plus petite note.

Une bonne habitude

Avant de programmer, se poser trois questions :

- Quelles sont les données en entrée ?
- Quelles étapes permettent de résoudre le problème ?
- Quel résultat doit-on obtenir ?

Minimum d'un tableau : l'algorithme

On dispose d'un tableau d'entiers. On souhaite trouver **l'indice de la plus petite valeur**.

Exemple

$$L = [12,\ 8,\ 15,\ 6,\ 10]$$

La plus petite valeur est 6, et son indice est 3.

Minimum d'un tableau : l'algorithme

On dispose d'un tableau d'entiers. On souhaite trouver **l'indice de la plus petite valeur**.

Exemple

$$L = [12,\ 8,\ 15,\ 6,\ 10]$$

La plus petite valeur est 6, et son indice est 3.

Algorithme

- **Entrée :** un tableau L d'entiers non vide.
- **Sortie :** l'indice du plus petit élément de L.

1. Initialiser m à 0 : on suppose que le premier élément est le minimum.
2. Parcourir les indices i de 1 à longueur$(L) - 1$.
3. Si $L[i] < L[m]$, alors mettre m à jour : $m \leftarrow i$.
4. Renvoyer m.

Minimum d'un tableau : en Python

Les étapes de l'algorithme se traduisent directement en Python :

Implémentation

```
def minimum(L):
    m = 0
    for i in range(1, len(L)):
        if L[i] < L[m]:
            m = i
    return m
```

Exemple d'utilisation

```
L = [12, 8, 15, 6, 10]
indice = minimum(L)
print(indice)     # 3
print(L[indice]) # 6
```

Question : comment adapter cet algorithme pour trouver l'indice du plus grand élément ?

Trier un tableau : le tri par sélection

On souhaite ranger les éléments d'un tableau dans l'ordre croissant.

Exemple

$$T = [5,\ 1,\ -8,\ 2,\ -4,\ 7]$$

On veut obtenir :

$$[-8,\ -4,\ 1,\ 2,\ 5,\ 7]$$

Trier un tableau : le tri par sélection

On souhaite ranger les éléments d'un tableau dans l'ordre croissant.

Exemple

$$T = [5,\ 1,\ -8,\ 2,\ -4,\ 7]$$

On veut obtenir :

$$[-8,\ -4,\ 1,\ 2,\ 5,\ 7]$$

Principe du tri par sélection

On construit progressivement la partie triée du tableau :

1. On cherche le plus petit élément du tableau.
2. On le place au début, en l'échangeant avec le premier élément.
3. On recommence avec la partie restante du tableau, en ignorant les éléments déjà placés.

Tri par sélection : déroulement

À chaque étape, on cherche le minimum dans la partie du tableau qui n'est pas encore triée, puis on l'échange avec le premier élément de cette partie.

Exemple : $T = [5,\ 1,\ -8,\ 2,\ -4,\ 7]$

- Départ : $[5,\ 1,\ -8,\ 2,\ -4,\ 7]$

Tri par sélection : déroulement

À chaque étape, on cherche le minimum dans la partie du tableau qui n'est pas encore triée, puis on l'échange avec le premier élément de cette partie.

Exemple : $T = [5,\ 1,\ -8,\ 2,\ -4,\ 7]$

- Départ : $[5,\ 1,\ -8,\ 2,\ -4,\ 7]$
- Étape 1 : minimum $= -8$ $[-8,\ 1,\ 5,\ 2,\ -4,\ 7]$

Tri par sélection : déroulement

À chaque étape, on cherche le minimum dans la partie du tableau qui n'est pas encore triée, puis on l'échange avec le premier élément de cette partie.

Exemple : $T = [5,\ 1,\ -8,\ 2,\ -4,\ 7]$

- Départ : $[5,\ 1,\ -8,\ 2,\ -4,\ 7]$
- Étape 1 : minimum $= -8$ $[-8,\ 1,\ 5,\ 2,\ -4,\ 7]$
- Étape 2 : minimum $= -4$ $[-8,\ -4,\ 5,\ 2,\ 1,\ 7]$

Tri par sélection : déroulement

À chaque étape, on cherche le minimum dans la partie du tableau qui n'est pas encore triée, puis on l'échange avec le premier élément de cette partie.

Exemple : $T = [5,\ 1,\ -8,\ 2,\ -4,\ 7]$

- Départ : $[5,\ 1,\ -8,\ 2,\ -4,\ 7]$
- Étape 1 : minimum $= -8$ $[-8,\ 1,\ 5,\ 2,\ -4,\ 7]$
- Étape 2 : minimum $= -4$ $[-8,\ -4,\ 5,\ 2,\ 1,\ 7]$
- Étape 3 : minimum $= 1$ $[-8,\ -4,\ 1,\ 2,\ 5,\ 7]$

Tri par sélection : déroulement

À chaque étape, on cherche le minimum dans la partie du tableau qui n'est pas encore triée, puis on l'échange avec le premier élément de cette partie.

Exemple : $T = [5,\ 1,\ -8,\ 2,\ -4,\ 7]$

- Départ : $[5,\ 1,\ -8,\ 2,\ -4,\ 7]$
- Étape 1 : minimum $= -8$ $[-8,\ 1,\ 5,\ 2,\ -4,\ 7]$
- Étape 2 : minimum $= -4$ $[-8,\ -4,\ 5,\ 2,\ 1,\ 7]$
- Étape 3 : minimum $= 1$ $[-8,\ -4,\ 1,\ 2,\ 5,\ 7]$
- Étapes suivantes : le tableau reste inchangé.

Tri par sélection : déroulement

À chaque étape, on cherche le minimum dans la partie du tableau qui n'est pas encore triée, puis on l'échange avec le premier élément de cette partie.

Exemple : $T = [5,\ 1,\ -8,\ 2,\ -4,\ 7]$

- Départ : $[5,\ 1,\ -8,\ 2,\ -4,\ 7]$
- Étape 1 : minimum $= -8$ $[-8,\ 1,\ 5,\ 2,\ -4,\ 7]$
- Étape 2 : minimum $= -4$ $[-8,\ -4,\ 5,\ 2,\ 1,\ 7]$
- Étape 3 : minimum $= 1$ $[-8,\ -4,\ 1,\ 2,\ 5,\ 7]$
- Étapes suivantes : le tableau reste inchangé.

À retenir

À chaque étape, un élément est placé définitivement à sa position correcte. La partie triée grandit progressivement.

Tri par sélection : l'algorithme

Algorithme

- **Entrée :** un tableau T.
- **Sortie :** le tableau T trié dans l'ordre croissant.

Pour chaque indice i de 0 à longueur$(T) - 2$:

1. Chercher l'indice m du minimum dans $T[i:]$.
2. Échanger $T[i]$ et $T[i+m]$.

Tri par sélection : l'algorithme

Algorithme

- **Entrée :** un tableau T.
- **Sortie :** le tableau T trié dans l'ordre croissant.

Pour chaque indice i de 0 à longueur$(T) - 2$:

1. Chercher l'indice m du minimum dans $T[i:]$.
2. Échanger $T[i]$ et $T[i+m]$.

Pourquoi $i + m$?

La recherche du minimum se fait dans le sous-tableau $T[i:]$. L'indice m est relatif à ce sous-tableau : on ajoute donc i pour obtenir l'indice correspondant dans le tableau initial.

Tri par sélection : en Python

On réutilise la fonction `minimum` vue précédemment.

Implémentation

```
def tri_selection(T):
    for i in range(len(T) - 1):
        m = minimum(T[i:])
        T[i], T[i + m] = T[i + m], T[i]
    return T
```

Exemple d'utilisation

```
T = [5, 1, -8, 2, -4, 7]
print(tri_selection(T))
# [-8, -4, 1, 2, 5, 7]
```

Construire un tableau trié progressivement

Une autre méthode consiste à construire le tableau trié **au fur et à mesure que l'on reçoit les éléments**.

Exemple

On reçoit successivement les nombres :

$$5, \quad 1, \quad -8, \quad 2, \quad -4, \quad 7$$

On veut construire un tableau trié dans l'ordre croissant.

Construire un tableau trié progressivement

Une autre méthode consiste à construire le tableau trié **au fur et à mesure que l'on reçoit les éléments**.

Exemple

On reçoit successivement les nombres :

$$5, \quad 1, \quad -8, \quad 2, \quad -4, \quad 7$$

On veut construire un tableau trié dans l'ordre croissant.

Principe : l'insertion

À chaque nouvel élément :

1. On cherche sa place dans le tableau déjà trié.
2. On décale les éléments suivants vers la droite pour libérer une place.
3. On insère le nouvel élément à sa bonne position.

Construire un tableau trié progressivement

Une autre méthode consiste à construire le tableau trié **au fur et à mesure que l'on reçoit les éléments**.

Exemple

On reçoit successivement les nombres :

$$5, \quad 1, \quad -8, \quad 2, \quad -4, \quad 7$$

On veut construire un tableau trié dans l'ordre croissant.

Principe : l'insertion

À chaque nouvel élément :

1. On cherche sa place dans le tableau déjà trié.
2. On décale les éléments suivants vers la droite pour libérer une place.
3. On insère le nouvel élément à sa bonne position.

Idée essentielle

Le tableau est trié après chaque insertion.

Insertion dans un tableau trié : exemple

Supposons que le tableau soit déjà trié :

$$T = [-8,\ 1,\ 2,\ 5]$$

On souhaite y insérer le nombre -4.

Insertion dans un tableau trié : exemple

Supposons que le tableau soit déjà trié :

$$T = [-8,\ 1,\ 2,\ 5]$$

On souhaite y insérer le nombre -4.

1. On parcourt le tableau jusqu'à trouver le premier élément supérieur ou égal à -4 : ici, 1.

Insertion dans un tableau trié : exemple

Supposons que le tableau soit déjà trié :

$$T = [-8,\ 1,\ 2,\ 5]$$

On souhaite y insérer le nombre -4.

1. On parcourt le tableau jusqu'à trouver le premier élément supérieur ou égal à -4 : ici, 1.
2. On décale les éléments à partir de cette position vers la droite.

Insertion dans un tableau trié : exemple

Supposons que le tableau soit déjà trié :

$$T = [-8,\ 1,\ 2,\ 5]$$

On souhaite y insérer le nombre -4.

1. On parcourt le tableau jusqu'à trouver le premier élément supérieur ou égal à -4 : ici, 1.
2. On décale les éléments à partir de cette position vers la droite.
3. On place -4 dans la case libérée.

Insertion dans un tableau trié : exemple

Supposons que le tableau soit déjà trié :

$$T = [-8,\ 1,\ 2,\ 5]$$

On souhaite y insérer le nombre -4.

1. On parcourt le tableau jusqu'à trouver le premier élément supérieur ou égal à -4 : ici, 1.
2. On décale les éléments à partir de cette position vers la droite.
3. On place -4 dans la case libérée.

Résultat

$$[-8,\ 1,\ 2,\ 5] \quad \longrightarrow \quad [-8,\ \boxed{-4},\ 1,\ 2,\ 5]$$

Insertion dans un tableau trié : algorithme

On suppose que le tableau T est déjà trié dans l'ordre croissant.

Algorithme

- **Entrée :** un tableau trié T et un élément t.
- **Sortie :** le tableau T avec t inséré à la bonne place.

1. Initialiser i à 0.
2. Tant que $i <$ longueur(T) et $T[i] < t$, avancer à l'indice suivant.
3. Décaler vers la droite les éléments à partir de l'indice i.
4. Placer t à l'indice i.

Insertion dans un tableau trié : algorithme

On suppose que le tableau T est déjà trié dans l'ordre croissant.

Algorithme

- **Entrée :** un tableau trié T et un élément t.
- **Sortie :** le tableau T avec t inséré à la bonne place.

1. Initialiser i à 0.
2. Tant que $i <$ longueur(T) et $T[i] < t$, avancer à l'indice suivant.
3. Décaler vers la droite les éléments à partir de l'indice i.
4. Placer t à l'indice i.

Comment trouver la position ?

On s'arrête dès que l'on rencontre un élément supérieur ou égal à t, ou lorsque l'on atteint la fin du tableau.

Construire un tableau trié : exemple complet

On insère successivement les éléments de la séquence : $5,\ 1,\ -8,\ 2,\ -4,\ 7$.

Évolution du tableau

- Insertion de 5 : [5]

Construire un tableau trié : exemple complet

On insère successivement les éléments de la séquence : $5,\ 1,\ -8,\ 2,\ -4,\ 7$.

Évolution du tableau

- Insertion de 5 : $[5]$
- Insertion de 1 : $[1,\ 5]$

Construire un tableau trié : exemple complet

On insère successivement les éléments de la séquence : $5,\ 1,\ -8,\ 2,\ -4,\ 7$.

Évolution du tableau

- Insertion de 5 : $[5]$
- Insertion de 1 : $[1,\ 5]$
- Insertion de -8 : $[-8,\ 1,\ 5]$

Construire un tableau trié : exemple complet

On insère successivement les éléments de la séquence : $5,\ 1,\ -8,\ 2,\ -4,\ 7$.

Évolution du tableau

- Insertion de 5 : $[5]$
- Insertion de 1 : $[1,\ 5]$
- Insertion de -8 : $[-8,\ 1,\ 5]$
- Insertion de 2 : $[-8,\ 1,\ 2,\ 5]$

Construire un tableau trié : exemple complet

On insère successivement les éléments de la séquence : $5,\ 1,\ -8,\ 2,\ -4,\ 7$.

Évolution du tableau

- Insertion de 5 : $[5]$
- Insertion de 1 : $[1,\ 5]$
- Insertion de -8 : $[-8,\ 1,\ 5]$
- Insertion de 2 : $[-8,\ 1,\ 2,\ 5]$
- Insertion de -4 : $[-8,\ -4,\ 1,\ 2,\ 5]$

Construire un tableau trié : exemple complet

On insère successivement les éléments de la séquence : $5,\ 1,\ -8,\ 2,\ -4,\ 7$.

Évolution du tableau

- Insertion de 5 : $[5]$
- Insertion de 1 : $[1,\ 5]$
- Insertion de -8 : $[-8,\ 1,\ 5]$
- Insertion de 2 : $[-8,\ 1,\ 2,\ 5]$
- Insertion de -4 : $[-8,\ -4,\ 1,\ 2,\ 5]$
- Insertion de 7 : $[-8,\ -4,\ 1,\ 2,\ 5,\ 7]$

Construire un tableau trié : exemple complet

On insère successivement les éléments de la séquence : $5,\ 1,\ -8,\ 2,\ -4,\ 7$.

Évolution du tableau

- Insertion de 5 : $[5]$
- Insertion de 1 : $[1,\ 5]$
- Insertion de -8 : $[-8,\ 1,\ 5]$
- Insertion de 2 : $[-8,\ 1,\ 2,\ 5]$
- Insertion de -4 : $[-8,\ -4,\ 1,\ 2,\ 5]$
- Insertion de 7 : $[-8,\ -4,\ 1,\ 2,\ 5,\ 7]$

À retenir

Après chaque insertion, le tableau reste trié. À la fin, tous les éléments ont été insérés.

Insertion dans un tableau trié : en Python

On utilise `insert(i, t)` pour insérer l'élément `t` à l'indice `i`. Les éléments suivants sont automatiquement décalés vers la droite.

Insérer un élément

```
def insertion(t, T):
    i = 0
    while i < len(T) and T[i] < t:
        i += 1
    T.insert(i, t)
```

Insertion dans un tableau trié : en Python

On utilise `insert(i, t)` pour insérer l'élément `t` à l'indice `i`. Les éléments suivants sont automatiquement décalés vers la droite.

Insérer un élément

```
def insertion(t, T):
    i = 0
    while i < len(T) and T[i] < t:
        i += 1
    T.insert(i, t)
```

Construire un tableau trié

```
T = []
for t in [5, 1, -8, 2, -4, 7]:
    insertion(t, T)

print(T)
# [-8, -4, 1, 2, 5, 7]
```

Suite de Fibonacci

On étudie une suite de nombres dans laquelle chaque terme, à partir du troisième, est obtenu en additionnant les deux termes précédents.

Définition

La suite de Fibonacci est définie par :

$$F_0 = 0, \qquad F_1 = 1$$

et, pour tout $n \geq 2$:

$$F_n = F_{n-1} + F_{n-2}$$

Suite de Fibonacci

On étudie une suite de nombres dans laquelle chaque terme, à partir du troisième, est obtenu en additionnant les deux termes précédents.

Définition

La suite de Fibonacci est définie par :

$$F_0 = 0, \qquad F_1 = 1$$

et, pour tout $n \geq 2$:

$$F_n = F_{n-1} + F_{n-2}$$

Premiers termes

n	0	1	2	3	4	5	6	7
F_n	0	1	1	2	3	5	8	13

Suite de Fibonacci

On étudie une suite de nombres dans laquelle chaque terme, à partir du troisième, est obtenu en additionnant les deux termes précédents.

Définition

La suite de Fibonacci est définie par :

$$F_0 = 0, \qquad F_1 = 1$$

et, pour tout $n \geq 2$:

$$F_n = F_{n-1} + F_{n-2}$$

Premiers termes

n	0	1	2	3	4	5	6	7
F_n	0	1	1	2	3	5	8	13

Objectif : calculer les termes de Fibonacci jusqu'au rang n.

Fibonacci : construire le tableau des termes

Pour calculer tous les termes jusqu'au rang n, on mémorise les valeurs dans un tableau.

Principe

1. Initialiser le tableau avec les deux premiers termes : $[0, 1]$.
2. Calculer le terme suivant en additionnant les deux derniers termes du tableau.
3. Ajouter ce nouveau terme au tableau.
4. Recommencer jusqu'au rang n.

Fibonacci : construire le tableau des termes

Pour calculer tous les termes jusqu'au rang n, on mémorise les valeurs dans un tableau.

Principe

1. Initialiser le tableau avec les deux premiers termes : $[0, 1]$.
2. Calculer le terme suivant en additionnant les deux derniers termes du tableau.
3. Ajouter ce nouveau terme au tableau.
4. Recommencer jusqu'au rang n.

Exemple : calcul jusqu'au rang 5

Étape	Tableau
Initialisation	$[0, 1]$
Ajout de F_2	$[0, 1, 1]$
Ajout de F_3	$[0, 1, 1, 2]$
Ajout de F_4	$[0, 1, 1, 2, 3]$
Ajout de F_5	$[0, 1, 1, 2, 3, 5]$

Fibonacci : calculer seulement le terme F_n

Pour calculer le terme suivant, on a besoin que des deux termes précédents.

Fibonacci : calculer seulement le terme F_n

Pour calculer le terme suivant, on a besoin que des deux termes précédents.

Nouvelle idée

Au lieu de mémoriser tout le tableau, on utilise deux variables :

- a : le terme précédent ;
- b : le terme courant.

À chaque étape, on calcule le terme suivant et on met à jour les deux variables : $(a, b) \leftarrow (b,\ a + b)$

Fibonacci : calculer seulement le terme F_n

Pour calculer le terme suivant, on a besoin que des deux termes précédents.

Nouvelle idée

Au lieu de mémoriser tout le tableau, on utilise deux variables :

- a : le terme précédent ;
- b : le terme courant.

À chaque étape, on calcule le terme suivant et on met à jour les deux variables : $(a, b) \leftarrow (b,\ a + b)$

Exemple : calcul de F_5

Étape	a	b
Départ	0	1
1	1	1
2	1	2
3	2	3
4	3	5
5	5	8

À la fin, $a = F_5 = 5$.

Suite de Fibonacci : en Python

On utilise deux variables pour calculer directement le terme demandé.

Implémentation

```
def fibonacci(n):
    a, b = 0, 1
    for i in range(n):
        a, b = b, a + b
    return a
```

Suite de Fibonacci : en Python

On utilise deux variables pour calculer directement le terme demandé.

Implémentation

```
def fibonacci(n):
    a, b = 0, 1
    for i in range(n):
        a, b = b, a + b
    return a
```

Exemples

```
print(fibonacci(5))   # 5
print(fibonacci(7))   # 13
```

Bilan : concevoir un algorithme efficace

Deux méthodes permettent de calculer les termes de Fibonacci :

- **Avec un tableau :** on mémorise tous les termes.
- **Avec deux variables :** on ne conserve que les deux derniers termes.

Bilan : concevoir un algorithme efficace

Deux méthodes permettent de calculer les termes de Fibonacci :

- **Avec un tableau :** on mémorise tous les termes.
- **Avec deux variables :** on ne conserve que les deux derniers termes.

À retenir

Un bon algorithme doit non seulement donner le bon résultat, mais aussi utiliser efficacement les ressources :

- limiter la mémoire utilisée ;
- limiter le nombre d'opérations effectuées.